

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN HUNTING SNAKE

Bộ môn: Kỹ thuật lập trình

Giảng viên lý thuyết: Trương Toàn Thịnh

Giảng viên thực hành: Mai Anh Tuấn

Nhóm thực hiện:

19127301: Trương Minh Trí

20127302: Đinh Anh Quốc

20127318: Phan Trí Tài

20127441: Võ Minh Anh

Hồ Chí Minh, ngày 14 tháng 04 năm 2022

Mục lục

1. Tổ chức thư mục trong chương trình	3
2. Những định nghĩa, biến được thêm vào	3
3. Xử lý va chạm và hiệu ứng khi va chạm.....	4
• <i>Hàm bool CrashGate(): rắn chạm phải cổng.....</i>	<i>4</i>
• <i>Hàm bool CrashItself(): rắn đụng chính nó</i>	<i>5</i>
• <i>Hàm void BlinkSnake(): rắn nhấp nháy khi chết.....</i>	<i>5</i>
4. Xử lý giao diện chính (menu) và hộp thoại thoát (exit)	6
• <i>Hàm void PrintLogo(): in logo khi bắt đầu chơi.....</i>	<i>6</i>
• <i>Hàm MENU menugame(): tạo menu bắt đầu trò chơi.....</i>	<i>7</i>
• <i>Hàm bool ContinueGame(): khi rắn chết có muốn chơi tiếp hay không</i>	<i>9</i>
5. Xử lý vẽ thức ăn, vẽ rắn (chuỗi MSSV)	11
• <i>Hàm void DrawFood()</i>	<i>11</i>
• <i>Hàm void DrawFood()</i>	<i>11</i>
• <i>Hàm void DrawSnake()</i>	<i>11</i>
• <i>Hàm void DrawSnake()</i>	<i>12</i>
6. Xử lý lưu/tải trò chơi (xử lý qua màn)	12
• <i>Hàm bool IsExistedFileName()</i>	<i>12</i>
• <i>Hàm bool IsValidFileName().....</i>	<i>12</i>
• <i>Hàm void DeleteBox()</i>	<i>13</i>
• <i>Hàm void SaveData()</i>	<i>13</i>
• <i>Hàm void LoadData()</i>	<i>16</i>
7. Xử lý giữ nguyên độ dài snake và khi ăn xong food ở một cấp	18
• <i>Hàm void DrawGate()</i>	<i>18</i>
• <i>Hàm void ProcessGate()</i>	<i>18</i>
• <i>Hàm void PrintWin()</i>	<i>20</i>
• <i>Hàm void PrintLose()</i>	<i>20</i>
8. Xử lý lưu top 5 HIGH LENGTH (chỉ xét các file data được lưu).....	21
• <i>Hàm bool IsEmptyHighLengthFile()</i>	<i>21</i>
• <i>Hàm void SaveHighLength()</i>	<i>22</i>

•	<i>Hàm void ResetHighLength()</i>	22
•	<i>Hàm void InitializeHighLength()</i>	23
•	<i>Hàm void CreateNewHighLength()</i>	23
•	<i>Hàm void SortHighLength()</i>	24
•	<i>Hàm void ShowHighLength()</i>	25
9.	Một số hàm được chỉnh sửa hoặc một số hàm phụ hỗ trợ	26
•	<i>Hàm void ThreadFunc()</i>	26
•	<i>Hàm int main()</i>	27
•	<i>Hàm void drawbox()</i>	29
•	<i>Hàm void FixConsoleWindow()</i>	29
•	<i>Hàm void InitialGame()</i>	29
•	<i>Hàm void ResetData()</i>	30
•	<i>Hàm void DrawBoard()</i>	30
•	<i>Hàm void drawbox()</i>	31
•	<i>Hàm void TextColor()</i>	31
•	<i>Hàm void ExitGame()</i>	32
10.	Nguồn tham khảo	32

1. Tổ chức thư mục trong chương trình

Thư mục Music, Data được lưu cùng cấp với file mã nguồn đồ án trong project.

Trong đó, Music chứa 3 file sound là: menu.wav, main.wav, dead.wav, win.wav để xử lý âm thanh khi đang ở menu, khi đang chơi và khi rắn chết.

Thư mục Data lưu file username.txt, highlength.txt và các file lưu dữ liệu người chơi.

- File username.txt chỉ đơn giản lưu tên các người chơi.
- File highlength.txt lưu mảng HighLength[5] (mỗi phần tử gồm tên và chiều dài) là 5 người lưu với chiều dài dài nhất. Mỗi dòng lưu lần lượt name và length của từng phần tử mảng HighLength[5].
- File dữ liệu người chơi có tên được lưu trong file username.txt, file này được tạo và tuân theo cấu trúc tuần tự trong hàm SaveData() (xem hàm SaveData() để biết cấu trúc).

2. Những định nghĩa, biến được thêm vào

`#pragma comment(lib, "winmm.lib")`: Để có thể add nhạc vào code.

`#pragma warning (disable: 4996)`: Để sử dụng các hàm nhập xuất, xử lý char* của C++ trong Visual Studio 2019

`#define MAX_SIZE_SNAKE 40`: chiều dài tối đa của rắn.

`#define MAX_SIZE_FOOD 4`: để qua được một round phải ăn 4 food.

`#define MAX_ROUND 9`: game có tất cả 9 round.

`#define ID_STUDENT "19127301201273022012731820127441"`: là chuỗi nối mã số sinh viên dùng làm cơ sở để vẽ snake ra màn hình.

Cấu trúc trên định nghĩa tên username và chiều dài snake tương ứng (chỉ xét các username được lưu).

```
struct HIGHLENGTH
{
    string name;
    int length;
};
```

Cấu trúc trên định nghĩa một menu tùy chọn, với thuộc tính sự lựa chọn và đang chọn cái nào.

```
struct MENU
{
    char* choice;
    char selecting;
```

```
};
```

HIGHLENGTH HighLength[5];: mảng lưu 5 người chơi có thành tích xuất sắc nhất.

HIGHLENGTH NewLength;

POINT a; :(anchor) điểm neo (điểm trung tâm) xác định vị trí của cổng khi qua màn.

int INDEX_ID;: chỉ ra kí tự cần lấy trong hằng chuỗi **ID_STUDENT** để vẽ thức ăn tương ứng với ký số tiếp theo.

int SIZE_PLUS;: phần bù cho lượng **SIZE_SNAKE** bị mất đi (xem hàm void ProcessGate() để dễ hiểu).

char newgame[] = "NEW GAME";: mảng ký tự lưu choice New Game.

char highlength[] = "HIGH SCORE";: mảng ký tự lưu choice High Length.

char exitgame[] = "EXIT GAME";: mảng ký tự lưu choice Exit Game. **char**

char yes[] = "Yes";: mảng ký tự lưu option Yes cho các MessageBox. **char**

char no[] = "No";: mảng ký tự lưu option No cho MessageBox. **char** ok[] =

char ok[] = "OK";: mảng ký tự lưu option OK cho MessageBox.

int temp;: ký tự điều khiển game (nhận vào các giá trị A, D, S, W...).

int win;: xác định rằng snake qua vòng.

3. Xử lý va chạm và hiệu ứng khi va chạm

- **Hàm bool CrashGate(): rắn chạm phải cổng**

Dòng 3, điểm “head” có tọa độ là đầu con rắn. Dòng 4-10, kiểm tra đầu con rắn có chạm cổng hay không, nếu đầu con rắn trùng một trong các điểm của cổng, ngoại trừ điểm giữa (a.x, a.y) theo chiều từ trên xuống (dòng 6) (tức khi con rắn chui vào cổng) thì hàm trả về true, ngược lại hàm trả về false.

Code
<pre>bool CrashGate() { POINT head = { snake[SIZE_SNAKE - 1].x, snake[SIZE_SNAKE - 1].y }; if ((head.x == a.x + 1 && head.y == a.y) (head.x == a.x - 1 && head.y == a.y - 1) (head.x == a.x && head.y == a.y && CHAR_LOCK == 'S') (head.x == a.x + 1 && head.y == a.y - 1) (head.x == a.x - 1 && head.y == a.y)) return true; return false;</pre>

```
}
```

- **Hàm bool *CrashItself()*: rắn đụng chính nó**

Dòng 3-5, vòng lặp for duyệt qua thân con rắn, mỗi lần thực hiện kiểm tra tọa độ điểm trên thân đó có trùng với tọa độ đầu con rắn hay không, nếu có thì hàm trả về true. Dòng 6, nếu chạy hết vòng lặp trên, đầu con rắn không chạm vào bất kỳ điểm nào trên thân nó thì hàm trả về false.

Code

```
bool CrashItself()
{
    for (int i = 0; i < SIZE_SNAKE - 1; i++)
        if (snake[i].x == snake[SIZE_SNAKE - 1].x && snake[i].y ==
snake[SIZE_SNAKE - 1].y)
            return true;
    return false;
}
```

- **Hàm void *BlinkSnake()*: rắn nhấp nháy khi chết**

Dòng 3 tạo một vòng lặp lớn bao quanh bên ngoài thể hiện số lần nhấp nháy của con rắn trong đó lồng thêm hai vòng lặp con (dòng 5, 11) chạy đến từng tọa độ trên thân con rắn in ra hình thù sau đó in ra khoảng trắng qua mỗi lần chạy vòng lặp lớn bên ngoài, trước mỗi vòng lặp con ta dùng hàm Sleep để dừng màn hình trong một khoảng thời gian nhất định để hiệu ứng nhấp nháy dễ nhận thấy hơn.

Code

```
void BlinkSnake()
{
    for (int j = 0; j < 5; j++)
    {
        Sleep(200);
        for (int i = 0; i < SIZE_SNAKE; i++)
        {
            GotoXY(snake[i].x, snake[i].y);
            cout << ID_STUDENT[i];
        }
        Sleep(100);
        for (int i = 0; i < SIZE_SNAKE; i++)
        {
            GotoXY(snake[i].x, snake[i].y);
            cout << " ";
        }
    }
}
```

4. Xử lý giao diện chính (menu) và hộp thoại thoát (exit)

- *Hàm void PrintLogo(): in logo khi bắt đầu chơi*

Khi khởi động trò chơi, chữ “SNAKE” sẽ hiện lên màn hình và đổi màu liên tục mỗi 70 miligiây (14 màu). Đầu tiên, dòng 3 và 4 xác định tọa độ bắt đầu in. Sau đó đến vòng lặp for (dòng 5-23), in từng dòng (x giữ nguyên, y lần lượt tăng thêm 1), mỗi dòng sẽ được đổi màu nhờ hàm TextColor().

```
Code

void PrintLogo()
{
    int xlogo = (WIDTH_CONSOLE / 2) - 37;
    int ylogo = (HEIGHT_CONSOLE / 2) - 10;

    for (int i = 1; i <= 14; i++)
    {
        char Line[] = "   _ _ _ _ _ _ _ _ _ _";
        GotoXY(xlogo, ylogo);
        TextColor(i, Line);

        strcpy(Line, "| | | | | | | \\ \\ | | _ _ | | \\ \\ | | / ___| / 
___|| \\ \\ | | / \\ \\ | | / / ___|");
        GotoXY(xlogo, ylogo + 1);
        TextColor(i, Line);

        strcpy(Line, "| | _ | | | | \\ \\ | | | | | | \\ \\ | | _ 
\\ \\ __ \\ \\ | \\ \\ | | / _ \\ \\ | ' / | _ | ");
        GotoXY(xlogo, ylogo + 2);
        TextColor(i, Line);

        strcpy(Line, "| _ _ | | _ | | \\ \\ | | | | | | \\ \\ | | _ | 
___ ) | \\ \\ | / _ \\ \\ | . \\ \\ | ___ ");
        GotoXY(xlogo, ylogo + 3);
        TextColor(i, Line);

        strcpy(Line, "| _ | _ \\ \\ _ _ / | _ \\ \\ | | _ | _ _ | _ \\ \\ | \\ \\ _ _ 
| _ _ / | _ \\ \\ _ / _ \\ \\ _ _ | \\ \\ _ _ _ _ ");
        GotoXY(xlogo, ylogo + 4);
        TextColor(i, Line);

        strcpy(Line, "   _ _ _ _ _ _ _ _ _ _ ");
        GotoXY(xlogo + 13, ylogo + 5);
        TextColor(i, Line);

        strcpy(Line, "| _ _ | _ _ | / \\ \\ | \\ \\ / | / _ \\ \\ _ _ |");
        GotoXY(xlogo + 13, ylogo + 6);
        TextColor(i, Line);

        strcpy(Line, " | | | _ | / _ \\ \\ | \\ \\ / | | | | / / ");
        GotoXY(xlogo + 13, ylogo + 7);
        TextColor(i, Line);

        strcpy(Line, " | | | | _ _ / _ _ \\ \\ | | | | | _ | / / ");
        GotoXY(xlogo + 13, ylogo + 8);
        TextColor(i, Line);

        strcpy(Line, "| _ | _ _ _ / _ \\ \\ _ _ | _ | \\ \\ _ _ // _ ");
```

```

        GotoXY(xlogo + 13, ylogo + 9);
        TextColor(i, Line);
        Sleep(70);
    }
}

```

• **Hàm MENU menugame(): tạo menu bắt đầu trò chơi**

Các biến xmenu, ymenu dùng để xác định các tọa độ của ba mảng chơi lên màn hình, biến MENU select sẽ liên tục nhận vào kí tự ‘w’, ‘s’ khi người chơi di chuyển lên xuống và giữ địa chỉ từng mảng ứng với từng điều kiện trong hàm khi mà người chơi vẫn chưa nhấn “enter”, cuối cùng khi kết thúc hàm select sẽ giữ địa chỉ mảng người chơi chọn và kí tự “enter”

(Dòng 12) mặc định ban đầu con trỏ sẽ giữ địa chỉ mảng “NEW GAME” do đó dòng NEW GAME sẽ được tô xanh và các dòng còn lại sẽ có màu trắng thông qua hàm TextColor.

(Dòng 13) Sau đó tạo một vòng lặp vô tận và trong vòng lặp kiểm tra các điều kiện như sau:

- ✦ Nếu con trỏ select đang giữ mảng “NEW GAME” thì khi người chơi nhấn “s” thì tô xanh “HIGH LENGTH” gán địa chỉ “HIGHs LENGTH” cho select
- ✦ Nếu con trỏ select đang giữ mảng “HIGH LENGTH”, thì khi người chơi nhấn “s” thì tô xanh “EXIT GAME” gán địa chỉ “EXIT GAME” cho select nếu nhấn “w” thì tô xanh “NEW GAME” và gán địa “NEW GAME” cho select
 - ✦ Nếu con trỏ đang giữ mảng “EXIT GAME” khi người chơi nhấn “w” thì tô xanh “HIGH LENGTH” và gán địa chỉ “HIGH_LENGTH” cho select
- ✦ Nếu tại một trong các điều kiện người chơi nhấn “enter” thì dừng vòng lặp và trả ra giá trị select, trong trường hợp (dòng 34) thì gọi hàm ShowHighlength () rồi dừng
- ✦ Các kí tự được lấy qua hàm _getch ().

Code
<pre> MENU menugame() { MENU select; int xmenu = (WIDTH_CONSOLE / 2) - 5; int ymenu = (HEIGH_CONSOLE / 2) + 3; PrintLogo(); for (int i = 0; i <= 2; ++i) drawbox(45, 15 + 3 * i, 16, 2); GotoXY(xmenu, ymenu); TextColor(3, newgame); </pre>


```

GotoXY(xmenu, ymenu + 3);
cout << highlength;
GotoXY(xmenu, ymenu + 6);
cout << exitgame;

select.choice = newgame;
while (true)
{
    if (select.choice == newgame)
    {
        select.selecting = _getch();
        if ((int)select.selecting == 13)
            break;
        if (select.selecting == 's')
        {
            GotoXY(xmenu, ymenu);
            TextColor(7, newgame);
            GotoXY(xmenu, ymenu + 3);
            TextColor(3, highlength);
            select.choice = highlength;
        }
    }
    if (select.choice == highlength)
    {
        select.selecting = _getch();
        if ((int)select.selecting == 13)
        {
            ShowHighLength();
            break;
        }
        if (select.selecting == 's')
        {
            GotoXY(xmenu, ymenu + 3);
            TextColor(7, highlength);
            GotoXY(xmenu, ymenu + 6);
            TextColor(3, exitgame);
            select.choice = exitgame;
        }
        if (select.selecting == 'w')
        {
            GotoXY(xmenu, ymenu + 3);
            TextColor(7, highlength);
            GotoXY(xmenu, ymenu);
            TextColor(3, newgame);
            select.choice = newgame;
        }
    }
    if (select.choice == exitgame)
    {
        select.selecting = _getch();
        if ((int)select.selecting == 13)
            break;
        if (select.selecting == 'w')
        {
            GotoXY(xmenu, ymenu + 6);
            TextColor(7, exitgame);
            GotoXY(xmenu, ymenu + 3);
            TextColor(3, highlength);
            select.choice = highlength;
        }
    }
}

```

```
        return select;
    }
```

- **Hàm bool ContinueGame(): khi rắn chết có muốn chơi tiếp hay không**

Các biến xgame, ygame, row, column dùng để xác định các tọa độ của bảng, dòng “Yes”, “No” (mảng char “Yes”, “No” là biến toàn cục) và dòng “Do you want to play more?”.

Biến bool Continue dùng để xác định người dùng chọn Yes hay No, biến MENU select dùng để giữ địa chỉ mảng “Yes” hoặc “No”, và nhận giá trị từ temp, biến int temp là biến toàn cục dùng để lấy các ký tự “enter”, “D”, “A” .

Dòng 36 Tạo hai vòng lặp lồng nhau để vẽ bản hình vuông qua ký tự 219 và 220.

Dòng 12 mặc định select sẽ giữ địa chỉ của “Yes” do đó dòng Yes sẽ được tô xanh và No được tô trắng bằng hàm TextColor do đó Continue mặc định là true, (dòng 38) tiếp theo kiểm tra xem biến temp có đang mang giá trị “D” hoặc “enter” hay không để tránh việc dùng **vòng lặp (cho phép người dùng di chuyển qua lại bằng “D”, “A” để chọn Yes hoặc No)** khi chưa chọn hoặc mảng “No” sẽ bị tô xanh đầu tiên.

(Dòng 40) sau đó tạo một lặp vô tận trong vòng lặp kiểm tra các điều kiện sau:

- ✦ Khi select đang giữ địa chỉ mảng “Yes” thì gán temp cho select và nếu select đó nhận vào “D” thì tô xanh mảng “No” và cho select giữ địa chỉ mảng tô xanh. ✦ Khi select đang giữ địa chỉ mảng “No” thì gán temp cho select và nếu select đó nhận vào “A” thì tô xanh mảng “Yes” và cho select giữ địa chỉ mảng tô xanh.
- ✦ Khi select nhận vào là “enter” thì dừng vòng lặp.
- ✦ Biến temp được lấy qua hàm toupper (_getch ()) trong main().

Sau khi xong vòng lặp kiểm tra điều kiện select đang giữ địa chỉ nào nếu là “Yes” thì gọi StartGame (), nếu là “No” thì gán Continue bằng false. Cuối cùng trả ra giá trị Continue.

Code

```
bool ContinueGame()
{
    bool Continue = true;

    int column = 30;
    int row = 7;
    int xgame = (WIDTH_CONSOLE / 2) - 15;
    int ygame = (HEIGHT_CONSOLE / 2) - 3;
    MENU select;

    GotoXY(food[FOOD_INDEX].x, food[FOOD_INDEX].y);
    cout << " ";

    for (int i = 0; i < row; i++)
```

```

{
    GotoXY(xgame, ygame + i);
    for (int j = 0; j < column; j++)
    {
        if (i == 0)
            cout << (unsigned char)220;
        else if (i == row - 1)
            cout << (unsigned char)223;
        else if (j == 0 || j == column - 1)
            cout << (unsigned char)219;
        else
            cout << " ";
    }
}
PrintLose();
GotoXY(xgame + 3, ygame + 2);
cout << "Do you want to play more?";
GotoXY(xgame + 8, ygame + 4);
TextColor(3, yes);
GotoXY(xgame + 20, ygame + 4);
cout << no;
select.choice = yes;

if (temp == 'D' || temp == '13') temp = 'A';

while (true)
{
    if (select.choice == yes)
    {
        select.selecting = temp;
        if ((int)select.selecting == 13)
            break;
        if (select.selecting == 'D')
        {
            GotoXY(xgame + 8, ygame + 4);
            TextColor(7, yes);
            GotoXY(xgame + 20, ygame + 4);
            TextColor(3, no);
            select.choice = no;
        }
    }
    if (select.choice == no)
    {
        select.selecting = temp;
        if ((int)select.selecting == 13)
            break;
        if (select.selecting == 'A')
        {
            GotoXY(xgame + 8, ygame + 4);
            TextColor(3, yes);
            GotoXY(xgame + 20, ygame + 4);
            TextColor(7, no);
            select.choice = yes;
        }
    }
}
if (select.choice == yes)
    StartGame();
if (select.choice == no)
    Continue = false;

return Continue;

```

```
}
```

5. Xử lý vẽ thức ăn, vẽ rắn (chuỗi MSSV)

- **Hàm void DrawFood()**

Nhóm đã tách hàm void DrawSnakeAndFood() có sẵn trong hướng dẫn thành ba hàm: void DrawFood(), void DrawSnake(), void DeleteTail() để dễ xử lý.

- **Hàm void DrawFood()**

Hàm DrawFood() đơn giản chỉ in ra 1 kí tự. Dòng 3 là xác định tọa độ của food. Dòng 4 là in ra kí tự ID_STUDENT[INDEX_ID], mỗi lần in ra là 1 con số ứng với con số của MSSV tiếp theo.

VD: Giả sử rắn có chiều dài 191204 thì khi ăn thêm food sẽ là 1912048 đúng với MSSV thì food sẽ hiện ra là số 8.

Code
<pre>void DrawFood() { GotoXY(food[FOOD_INDEX].x, food[FOOD_INDEX].y); cout << ID_STUDENT[INDEX_ID]; }</pre>

- **Hàm void DrawSnake()**

Hàm DrawSnake() là in ra các kí tự trong mảng toàn cục ID_STUDENT[] từ vị trí 0 đến nhỏ hơn SIZE_SNAKE. Mỗi khi chạy 1 vòng lặp for(dòng 3) thì sẽ tạo 1 biến kiểu char (dòng 5) gán bằng ID_STUDENT[i] (kí tự tại vị trí i), dòng 6 là xác định tọa độ để vẽ và dòng 7 để in ra kí tự đó.

Code
<pre>void DrawSnake() { for (int i = SIZE_SNAKE; i >= 0; i--) { char body = ID_STUDENT[SIZE_SNAKE - i - 1]; GotoXY(snake[i].x, snake[i].y); cout << body; } }</pre>

- **Hàm void DrawSnake()**

Hàm DeleteTail() dùng để xóa phần đuôi thừa khi con rắn dịch tới một bước. Mỗi khi chạy 1 vòng lặp (dòng 3) thì dòng 5 xác định vị trí kí tự đó và dòng 6 xóa kí tự bằng cách in ra khoảng trắng tại vị trí đó.

Code
<pre>void DeleteTail() { for (int i = 0; i < SIZE_SNAKE; i++) { GotoXY(snake[i].x, snake[i].y); cout << " "; } }</pre>

6. Xử lý lưu/tải trò chơi (xử lý qua màn)

- **Hàm bool IsExistedFileName()**

Hàm IsExistedFileName dùng để kiểm tra tên FileName đã tồn tại hay chưa bằng cách đọc tên các file lưu trên từng dòng trong file username.txt (dòng 6: f_user >> tmp) rồi so sánh với FileName (dòng 7). Hàm trả về true nếu tên FileName đã tồn tại và ngược lại.

Code
<pre>bool IsExistedFileName(string FileName) { ifstream f_user("..\Data\username.txt"); string tmp; while (f_user >> tmp) if (tmp == FileName) { f_user.close(); return true; } f_user.close(); return false; }</pre>

- **Hàm bool IsValidFileName()**

Hàm IsValidFileName dùng để kiểm tra tên FileName có hợp lệ hay không (không chứa các kí tự > < : " / \ | ? *) bằng cách kiểm tra từng kí tự của FileName trong vòng lặp for.

Hàm trả về false nếu tên FileName chứa kí tự không hợp lệ và ngược lại.

Code
<pre>bool IsValidFileName(string FileName) { for (int i = 0; i < FileName.length(); i++) if (FileName[i] == '>' FileName[i] == '<' FileName[i] == ':' FileName[i] == '"' FileName[i] == '/' FileName[i] == '\\ ' FileName[i] == ' ' FileName[i] == '?' FileName[i] == '*') return false; return true; }</pre>

- **Hàm void DeleteBox()**

Hàm DeleteBox() dùng để xóa cái hộp thoại. Từ dòng 3 đến dòng 6 là khai báo tọa độ chính cần xóa. Từ dòng 8 đến dòng 21 là để xóa hộp thoại bằng cách các vị trí chứa các kí tự tạo thành khung bảng in thành các khoảng trắng.

Code
<pre>void DeleteBox() { int column = 30; int row = 8; int xgame = (WIDTH_CONSOLE / 2) - 15; int ygame = (HEIGHT_CONSOLE / 2) - 3; for (int i = 0; i < row; i++) { GotoXY(xgame, ygame + i); for (int j = 0; j < column; j++) { if (i == 0) cout << " "; else if (i == row - 1) cout << " "; else if (j == 0 j == column - 1) cout << " "; else cout << " "; } } }</pre>

- **Hàm void SaveData()**

Các biến column, row, xgame, ygame dùng để xác định tọa độ của hộp thoại Save Data, vòng lặp for để vẽ hộp thoại.

Sau đó, kiểm tra điều kiện khi nhập tên file vào biến FileName, hàm IsExistedFileName() kiểm tra tên file nhập có trùng hay chưa và hàm

IsValidFileName() kiểm tra tính hợp lệ của tên file. Người chơi sẽ nhập lại FileName cho đến khi thỏa. Hàm DeleteBox() xử lý xóa hộp thoại SaveData sau khi lưu thành công.

Sau đó tạo file lưu với tên vừa nhập trong thư mục Data (đường dẫn tương đối). Ta cần có một file để lưu tên tất cả các file đã được lưu để dễ duyệt lại sau này, biến f_user đại diện cho username.txt với chế độ ios::app. Lần lượt lưu từng thuộc tính game vào file, theo thứ tự SIZE_SNAKE, SIZE_PLUS.

Với SIZE_SNAKE dòng tiếp theo, lưu tọa độ từng đốt của snake. Tiếp theo, lưu vào FOOD_INDEX, food[FOOD_INDEX].x, food[FOOD_INDEX].y.

Nếu tồn tại cổng (GATE_EXIST) thì lưu vào tọa độ điểm neo của cổng, ngược lại chỉ xuất ra -1 và -1 để dễ đánh dấu. Các dòng tiếp theo lưu vào INDEX_ID, SPEED, ROUND, MOVING, CHAR_LOCK.

Dòng 88, lưu tên và chiều dài vào NewLength. CreateNewHighLength() thực hiện lưu NewLength vào top 5 High Length nếu độ dài lọt top. Dòng 92 hàm SortHighLength() dùng để sắp xếp mảng HighLength[5] theo thứ tự length giảm dần.

Code
<pre>void SaveData() { string FileName; int column = 30; int row = 8; int xgame = (WIDTH_CONSOLE / 2) - 15; int ygame = (HEIGHT_CONSOLE / 2) - 3; for (int i = 0; i < row; i++) { GotoXY(xgame, ygame + i); for (int j = 0; j < column; j++) { if (i == 0) cout << (unsigned char)220; else if (i == row - 1) cout << (unsigned char)223; else if (j == 0 j == column - 1) cout << (unsigned char)219; else cout << " "; } } GotoXY(xgame + 9, ygame + 2); cout << "Save and Exit"; GotoXY(xgame + 14, ygame + 5); TextColor(3, ok); GotoXY(xgame + 3, ygame + 3); cout << "Name: "; do {</pre>

```

        GotoXY(xgame + 9, ygame + 3);
        cin >> FileName;
        GotoXY(xgame + 3, ygame + 4);
        if (IsExistedFileName(FileName))
            cout << "File existed, re-type!";
        if (!IsValidFileName(FileName))
            cout << "Invalid char, re-type!";
        if (FileName.length() > 18)
            cout << "Too long, re-type!";

        if (IsExistedFileName(FileName) || !IsValidFileName(FileName) ||
        FileName.length() > 18)
        {
            GotoXY(xgame + 9, ygame + 3);
            for (int i = 0; i < 18; i++)
                cout << " ";
        }
    } while (IsExistedFileName(FileName) || !IsValidFileName(FileName) ||
    FileName.length() > 18);

    DeleteBox();

    ofstream fo("Data\\" + FileName);

    ofstream f_user;
    f_user.open("Data\\username.txt", ios::app);
    f_user << FileName << endl;
    f_user.close();

    fo << SIZE_SNAKE << " " << SIZE_PLUS << endl;

    for (int i = 0; i < SIZE_SNAKE; i++)
        fo << snake[i].x << " " << snake[i].y << endl;

    fo << FOOD_INDEX << endl;

    fo << food[FOOD_INDEX].x << " " << food[FOOD_INDEX].y << endl;

    if (GATE_EXIST)
        fo << a.x << " " << a.y << endl;
    else
        fo << -1 << " " << -1 << endl;

    fo << INDEX_ID << endl;

    fo << SPEED << endl;

    fo << ROUND << endl;

    fo << MOVING << endl;

    fo << CHAR_LOCK << endl;

    fo << win;

    fo.close();

    NewLength.name = FileName;
    NewLength.length = SIZE_SNAKE + SIZE_PLUS;

    CreateNewHighLength();
    SortHighLength();

```



```
}
```

- **Hàm void LoadData()**

Phân tạo hộp thoại giống như trong hàm SaveData(). Riêng ở phần nhập FileName, tiếp tục sử dụng hàm IsExistedFileName(), nhưng lần này người dùng phải nhập cho đến khi tìm được tên tương ứng trong file username.txt. Ta cần phải xóa “snake cũ” trên màn hình, nên dùng vòng for kết hợp với hàm GotoXY() để cout ký tự trắng tại các điểm này.

Sau đó mở file vào lần lượt tải các thuộc tính game vào, bắt đầu với SIZE_SNAKE, SIZE_PLUS, cần phải cập nhật lại chiều dài rắn ở màn hình chơi, nên ta cout << SIZE_SNAKE + SIZE_PLUS. Với SIZE_SNAKE dòng tiếp theo, tải vào tọa độ của snake. Tiếp theo ta tải vào FOOD_INDEX, food[FOOD_INDEX].x, food[FOOD_INDEX].y.

Sau đó tải vào tọa độ điểm neo của cổng, nếu tồn tại cổng (hai tọa độ khác -1) thì thực hiện vẽ cổng và gán GATE_EXIST = true. Các dòng tiếp theo tải vào INDEX_ID, SPEED, ROUND, MOVING, CHAR_LOCK.

Code

<pre>void LoadData() { string FileName; int column = 30; int row = 8; int xgame = (WIDTH_CONSOLE / 2) - 15; int ygame = (HEIGHT_CONSOLE / 2) - 3; for (int i = 0; i < row; i++) { GotoXY(xgame, ygame + i); for (int j = 0; j < column; j++) { if (i == 0) cout << (unsigned char)220; else if (i == row - 1) cout << (unsigned char)223; else if (j == 0 j == column - 1) cout << (unsigned char)219; else cout << " "; } } GotoXY(xgame + 11, ygame + 2); cout << "Load data"; GotoXY(xgame + 14, ygame + 5); TextColor(3, ok); GotoXY(xgame + 3, ygame + 3); cout << "Name: "; do</pre>
--

```

{
    GotoXY(xgame + 9, ygame + 3);
    cin >> FileName;
    if (!IsExistedFileName(FileName))
    {
        GotoXY(xgame + 3, ygame + 4);
        cout << "Not existed user!";
        GotoXY(xgame + 9, ygame + 3);
        for (int i = 0; i < 18; i++)
            cout << " ";
    }
} while (!IsExistedFileName(FileName));

DeleteBox();

for (int i = 0; i < SIZE_SNAKE; i++)
{
    GotoXY(snake[i].x, snake[i].y);
    cout << " ";
}

ifstream fi(".\\Data\\" + FileName);

fi >> SIZE_SNAKE >> SIZE_PLUS;

GotoXY(8, HEIGH_CONSOLE);
cout << SIZE_SNAKE + SIZE_PLUS;

for (int i = 0; i < SIZE_SNAKE; i++)
    fi >> snake[i].x >> snake[i].y;

GotoXY(food[FOOD_INDEX].x, food[FOOD_INDEX].y);

fi >> FOOD_INDEX;
cout << " ";
fi >> food[FOOD_INDEX].x >> food[FOOD_INDEX].y;

fi >> a.x >> a.y;
if (a.x != -1 && a.y != -1)
{
    GotoXY(a.x + 1, a.y);
    cout << (unsigned char)223;
    GotoXY(a.x - 1, a.y - 1);
    cout << (unsigned char)219;
    GotoXY(a.x, a.y);
    cout << (unsigned char)223;
    GotoXY(a.x + 1, a.y - 1);
    cout << (unsigned char)219;
    GotoXY(a.x - 1, a.y);
    cout << (unsigned char)223;
    GATE_EXIST = true;
}

fi >> INDEX_ID;

fi >> SPEED;

fi >> ROUND;

fi >> MOVING;

fi >> CHAR_LOCK;

```

```

        fi >> win;

        fi.close();
    }

```

7. Xử lý giữ nguyên độ dài snake và khi ăn xong food ở một cấp

• Hàm void DrawGate()

Dòng 3 – 8: Hàm DrawGate sẽ thực hiện random tọa độ điểm chính giữa của cổng sao cho điểm này và các điểm còn lại của cổng không trùng với con rắn.

Sau đó hàm sẽ thực hiện vẽ cổng tại các điểm này. Hàm gán biến GATE_EXIST = true có nghĩa là cổng đã được tạo và đang hiện trên màn hình.

Code

```

void DrawGate()
{
    do
    {
        a.x = rand() % (WIDTH_CONSOLE - 22) + 2;
        a.y = rand() % (HEIGHT_CONSOLE - 4) + 3;
    } while (!IsValid(a.x, a.y) || !IsValid(a.x - 1, a.y) || !IsValid(a.x + 1, a.y) || !IsValid(a.x - 1, a.y - 1) || !IsValid(a.x + 1, a.y - 1));

    GotoXY(a.x + 1, a.y);
    cout << (unsigned char)223;
    GotoXY(a.x - 1, a.y - 1);
    cout << (unsigned char)219;
    GotoXY(a.x, a.y);
    cout << (unsigned char)223;
    GotoXY(a.x + 1, a.y - 1);
    cout << (unsigned char)219;
    GotoXY(a.x - 1, a.y);
    cout << (unsigned char)223;

    GATE_EXIST = true;
}

```

• Hàm void ProcessGate()

Ta đặt hàm này trong ThreadFunc().

Đầu tiên, nếu cổng không tồn tại thì cứ tạo food bình thường. Nếu tồn tại cổng và tọa độ đầu snake trùng với tọa độ điểm neo thì cho chiều dài của snake giảm dần mỗi khi chui vào cổng và SIZE_PLUS thì tăng dần để “bù đắp” lại lượng SIZE_SNAKE giảm.

Sau đó ta cập nhật lại giá trị ROUND ngoài màn hình game. Cho đến khi SIZE_SNAKE == 0 (snake chui hẳn vào cổng) thì ta thực hiện xóa cổng, gán GATE_EXIST = false, tăng vòng chơi, tốc độ, vì lúc này SIZE_SNAKE == 0 nên ta cần gán SIZE_SNAKE = 4 * ROUND để cập nhật lại chiều dài thực.

Vòng for tiếp theo dùng để gán từng tọa độ của snake lại bằng tọa độ điểm neo vì khi SIZE_SNAKE-- thì các giá trị cuối có thể bị rác, nên ta cần tái gán lại. Cuối cùng ta cho rắn chui ra cổng bằng cách gán tung độ đầu rắn bằng tung độ điểm neo cộng 1 (chui ra nên cộng).

Code

```
void ProcessGate()
{
    if (!GATE_EXIST)
        DrawFood();
    if (snake[SIZE_SNAKE - 1].x == a.x && snake[SIZE_SNAKE - 1].y == a.y &&
GATE_EXIST && CHAR_LOCK == 'W')
    {
        SIZE_SNAKE--;
        SIZE_PLUS++;
    }

    GotoXY(WIDTH_CONSOLE - 6, 3);
    cout << ROUND;

    if (SIZE_SNAKE == 0)
    {
        GotoXY(a.x + 1, a.y);
        cout << " ";
        GotoXY(a.x - 1, a.y - 1);
        cout << " ";
        GotoXY(a.x, a.y);
        cout << " ";
        GotoXY(a.x + 1, a.y - 1);
        cout << " ";
        GotoXY(a.x - 1, a.y);
        cout << " ";

        GATE_EXIST = false;

        win++;
        ROUND++;
        SPEED++;
        SIZE_SNAKE = 4 * ROUND;
        SIZE_PLUS = 0;

        for (int i = 0; i < SIZE_SNAKE; i++)
        {
            snake[i].x = a.x;
            snake[i].y = a.y;
        }

        snake[SIZE_SNAKE - 1].x = a.x;
        snake[SIZE_SNAKE - 1].y = a.y + 1;
    }
}
```

```
}
```

- **Hàm void PrintWin()**

Hàm PrintWin() dùng để vẽ bảng chữ “ WIN” khi người chơi chiến thắng 9 màn. Dòng 5 dùng để xóa màn hình hiện tại, dòng 27 là dừng màn hình với 72ms. Các lệnh TextColor(1, Line1) dùng để in mảng Line1 có màu. Dòng 25, 26 dùng để gán i để in liên tục các màu trong lệnh TextColor.

Code

<pre>void PrintWin() { int xwinner = (WIDTH_CONSOLE / 2) - 12; int ywinner = (HEIGHT_CONSOLE / 2) - 3; system("cls"); int i = 1; while (true) { char Line1[] = " __ ----- - - "; GotoXY(xwinner, ywinner); TextColor(i, Line1); strcpy(Line1, " \\ \\ \\ \\ / / _ \\ \\ "); GotoXY(xwinner, ywinner + 1); TextColor(i, Line1); strcpy(Line1, " \\ \\ \\ \\ / \\ / / \\ \\ "); GotoXY(xwinner, ywinner + 2); TextColor(i, Line1); strcpy(Line1, " \\ \\ v v / \\ \\ "); GotoXY(xwinner, ywinner + 3); TextColor(i, Line1); strcpy(Line1, " \\ \\ _ / \\ \\ _ / _ _ \\ \\ _ "); GotoXY(xwinner, ywinner + 4); TextColor(i, Line1); i++; if (i == 14) i = 1; Sleep(72); } }</pre>

- **Hàm void PrintLose()**

Hàm PrintWin() dùng để vẽ bảng chữ “ LOSE” khi người chơi va chạm vào tường, công, tự ăn chính mình. Hàm này sẽ khá giống với với hàm PrintWin().

Code

<pre>void PrintLose() { int xwinner = (WIDTH_CONSOLE / 2) - 12; int ywinner = (HEIGHT_CONSOLE / 2) - 3; system("cls"); int i = 1; int x = 1; while (true)</pre>

```

{
    char Line1[] = " _ _ _ _ _ ";
    GotoXY(xwinner, ywinner);
    TextColor(i, Line1);
    strcpy(Line1, " | | / _ \\ \\ _ _ | | _ _ |");
    GotoXY(xwinner, ywinner + 1);
    TextColor(i, Line1);
    strcpy(Line1, " | | | | | \\ \\ _ _ \\ \\ | _ | ");
    GotoXY(xwinner, ywinner + 2);
    TextColor(i, Line1);
    strcpy(Line1, " | | _ | | _ | _ _ ) | | _ _ ");
    GotoXY(xwinner, ywinner + 3);
    TextColor(i, Line1);
    strcpy(Line1, " | _ _ _ \\ \\ _ _ / | _ _ _ / | _ _ _ |");
    GotoXY(xwinner, ywinner + 4);
    TextColor(i, Line1);
    i++;
    if (i == 14)
        i = 1;
    Sleep(72);
    ++x;
    if (x == 30)
        break;
}
system("cls");
}

```

8. Xử lý lưu top 5 HIGH LENGTH (chỉ xét các file data được lưu)

• Hàm *bool IsEmptyHighLengthFile()*

Hàm IsEmptyHighLengthFile thực hiện kiểm tra file highlength.txt có rỗng hay không.

Dòng 8: fi >> name >> length thực hiện đọc tên lưu vào name và chiều dài lưu vào length ở dòng đầu của highlength.txt. Nếu đọc-lưu được thì nghĩa là file đã có dữ liệu (không rỗng), trả về false và ngược lại.

Code
<pre> bool IsEmptyHighLengthFile() { ifstream fi; string name; int length; fi.open("Data\\highlength.txt"); if (fi >> name >> length) { fi.close(); return false; } fi.close(); return true; } </pre>

- **Hàm void SaveHighLength()**

Hàm SaveHighLength() dùng để cập nhật thông tin top 5 người chơi với chiều dài con rắn dài nhất (mảng HighLength[5]) vào file highlength.txt.

Dòng 3: remove(".\\Data\\highlength.txt"); dùng để xóa đi file highlength.txt lưu các thông tin của các người chơi của top 5 trước đó mỗi khi muốn cập nhật.

Dòng 6: Tạo file highlength.txt mới và mở file.

Dòng 8 – 12: Lưu top 5 hiện tại trên 5 dòng đầu của highlength.txt, mỗi dòng lưu thông tin tên và chiều dài con rắn của người chơi, cách nhau bởi khoảng trắng " " rồi đóng file.

Code
<pre>void SaveHighLength() { remove(".\\Data\\highlength.txt"); ofstream fo; fo.open(".\\Data\\highlength.txt"); for (int i = 0; i < 4; i++) fo << HighLength[i].name << " " << HighLength[i].length << endl; fo << HighLength[4].name << " " << HighLength[4].length; fo.close(); }</pre>

- **Hàm void ResetHighLength()**

Thực hiện việc gán tất cả các phần tử mảng HighLength với thông tin name mặc định là "[NONE]" và length mặc định là 4. Gọi hàm SaveHighLength (dòng 8) để cập nhật thông tin vừa high length vừa gán vào highlength.txt.

Code
<pre>void ResetHighLength() { for (int i = 0; i < 5; i++) { HighLength[i].name = "[NONE]"; HighLength[i].length = 4; } SaveHighLength(); }</pre>

- **Hàm void InitializeHighLength()**

Hàm InitializeHighLength dùng để khởi tạo thông tin top 5 highlength khi bắt đầu chạy chương trình.

if (!IsEmptyHighLengthFile()): nếu file highlength.txt có dữ liệu top 5 thì thực hiện đọc file theo từng dòng fi >> name >> length và lưu các thông tin top 5 vào mảng HighLength[5]. Nếu file highlength.txt rỗng thì dùng hàm ResetHighLength ở trên để tạo các giá trị mặc định cho mảng HighLength[5] và lưu vào file.

Code
<pre>void InitializeHighLength() { if (!IsEmptyHighLengthFile()) { string name; int length; ifstream fi; fi.open(".\\Data\\highlength.txt"); int i = 0; while (fi >> name >> length) { HighLength[i].name = name; HighLength[i].length = length; i++; } fi.close(); } else ResetHighLength(); }</pre>

- **Hàm void CreateNewHighLength()**

Hàm CreateNewHighLength được đặt cuối hàm SaveData sau khi lưu thông tin tên và chiều dài vào biến NewLength.

Dòng 3 – 19: tìm phần tử HighLength với giá trị length nhỏ nhất minLength trong top 5. (nếu tên là [NONE] thì mặc định là nhỏ nhất, thoát khỏi vòng lặp for). Biến index dùng để lưu chỉ số của phần tử có length nhỏ nhất.

Dòng 20 – 24: thực hiện cập nhật thông tin NewLength vào phần tử có length là minLength và chỉ số index nếu NewLength.length lọt vào top 5 chiều dài (Thứ tự so sánh xếp hạng: Phần tử lớn hơn > Phần tử lưu trước > Phần tử { [NONE], 4 }).

Code
<pre>void CreateNewHighLength() { int minLength = HighLength[0].length;</pre>


```

int index = 0;
for (int i = 0; i < 5; i++)
{
    if (HighLength[i].name == "[NONE]")
    {
        minLength = HighLength[i].length;
        index = i;
        break;
    }
    else if (HighLength[i].length < minLength)
    {
        minLength = HighLength[i].length;
        index = i;
    }
}

if (NewLength.length > minLength || (NewLength.length == 4 &&
HighLength[index].name == "[NONE]"))
{
    HighLength[index].name = NewLength.name;
    HighLength[index].length = NewLength.length;
}
}

```

• *Hàm void SortHighLength()*

Hàm SortHighLength dùng để sắp xếp các phần tử mảng HighLength[5] theo thứ tự giảm dần chiều dài và cập nhật vào highlength.txt.

Dòng 3 – 13: Chạy 2 vòng lặp for để so sánh chiều dài các phần tử trong mảng HighLength[5]. Nếu chiều dài phần tử sau lớn hơn phần tử trước thì thực hiện đổi thông tin tên name và chiều dài length cho nhau thông qua các biến tạm.

SaveHighLength: cập nhật lại thông tin top 5 sau khi sắp xếp vào highlength.txt.

Code
<pre> void SortHighLength() { for (int i = 0; i < 4; i++) for (int j = i + 1; j < 5; j++) if (HighLength[i].length < HighLength[j].length) { string name = HighLength[i].name; HighLength[i].name = HighLength[j].name; HighLength[j].name = name; int length = HighLength[i].length; HighLength[i].length = HighLength[j].length; HighLength[j].length = length; } SaveHighLength(); } </pre>

- **Hàm void ShowHighLength()**

Hàm ShowHighLength dùng để xuất các thông tin tên và chiều dài con rắn của top 5 High Length theo thứ tự giảm dần chiều dài con rắn:

Dòng 9: system("cls") xóa màn hình console để chuẩn bị vẽ giao diện HIGH LENGTH

Dòng 11- 32: thực hiện vẽ bảng HIGH LENGTH thông qua các tọa độ xHighLength, yHighLength,...

Dòng 34 – 53: đọc các thông tin name và length của top 5 High Length đã được lưu trong file highlength.txt và in ra màn hình trong phạm vi bảng HIGH LENGTH vừa vẽ (ngoài ra còn in thêm # i + 1 chính là xếp hạng theo chiều dài con rắn của top 5. (Do thông tin lưu trong file highlength.txt đã được sắp xếp theo thứ tự giảm dần chiều dài thông qua hàm SortHighLength).

Dòng 52 – 53: bảng xếp hạng HIGH LENGTH sẽ tắt đi và thoát chương trình khi ta nhấn 1 phím bất kì (dùng if (_kbhit()) break; để thoát khỏi vòng lặp while(true));

Code
<pre>void ShowHighLength() { ifstream fi; fi.open(".\\Data\\highlength.txt"); string name; int length; system("cls"); int column = 31; int row = 9; int xHighLength = (WIDTH_CONSOLE / 2) - 15; int yHighLength = (HEIGHT_CONSOLE / 2) - 4; for (int j = 0; j < row; j++) { GotoXY(xHighLength, yHighLength + j); for (int k = 0; k < column; k++) { if (j == 0) cout << (unsigned char)220; else if (j == 2 && k != 0 && k != column - 1) cout << "*"; else if (j == row - 1) cout << (unsigned char)223; else if (k == 0 k == column - 1) cout << (unsigned char)219; else cout << " "; } } while (true) {</pre>

```

GotoXY(xHighLength + 10, yHighLength + 1);
cout << "HIGH SCORE";

int i = 0;

while (fi >> name >> length)
{
    GotoXY(xHighLength + 4, yHighLength + i + 3);
    cout << "#" << i + 1 << ". ";
    GotoXY(xHighLength + 8, yHighLength + i + 3);
    cout << name;
    GotoXY(xHighLength + 27, yHighLength + i + 3);
    cout << length;
    i++;
}

if (_kbhit())
    break;
}
fi.close();
}

```

9. Một số hàm được chỉnh sửa hoặc một số hàm phụ hỗ trợ

• Hàm void ThreadFunc()

Trong hàm này có vòng lặp vô tận, nếu STATE == 1 tức thì mới tiếp tục kiểm tra biến MOVING xử lý di chuyển, ngược lại STATE == 0 như vậy vòng lặp sẽ không làm gì. Trong trường hợp rắn vẫn còn sống, các hàm DeleteTail(), ProcessGate(), DrawSnake() được tuần tự kiểm tra và thực hiện, nếu rắn chết, hàm ContinueGame() được gọi và hỏi người chơi có muốn chơi tiếp hay không.

Code
<pre> void ThreadFunc() { while (true) { if (STATE == 1) { DeleteTail(); switch (MOVING) { case 'A': CHAR_LOCK = 'D'; MoveLeft(); break; case 'D': CHAR_LOCK = 'A'; MoveRight(); break; case 'W': CHAR_LOCK = 'S'; MoveUp(); break; case 'S': CHAR_LOCK = 'W'; </pre>

```

        MoveDown();
        break;
    }
    ProcessGate();
    if (win == 9) {
        system("cls");
        PrintWin();
    }
    else
        DrawSnake();
    Sleep(200 / SPEED);
}
else
{
    bool Continue = ContinueGame();
    if (!Continue)
    {
        system("cls");
        exit(0);
    }
}
}
}

```

• Hàm *int main()*

Ban đầu, gọi các hàm `InitialGame()`, `InitializeHighLength()` để khởi tạo game và khởi tạo xếp hạng chiều dài snake, hàm `PlaySound` dùng chèn nhạc nền menu cho game, sau đó tạo một menu. Vòng lặp ở dòng 8 cho phép người dùng chuyển đổi giữa Play, High Game và Exit.

Nếu `STATE == 1` thì game bắt đầu, chương trình tạo một thread `t1` mới với tham số vào là hàm `ThreadFunc`. Vòng lặp ở dòng 29, if đầu tiên so sánh `win == 9` báo hiệu đã chơi hết màn và in ra chiến thắng, các if sau xử lý ký tự đã nhận từ bàn phím và điều khiển chương trình (P là tạm dừng, L là lưu, T là tải, Esc là buộc dừng).

Code
<pre> int main() { InitialGame(); InitializeHighLength(); PlaySound(L".\\Music\\menu.wav", NULL, SND_FILENAME SND_LOOP SND_ASYNC); MENU choose = menugame(); while (true) { if (choose.selecting == 13) { if (choose.choice == highlength choose.choice == exitgame) { STATE = 0; system("cls"); break; } } } } </pre>

```

        }
        system("cls");
        STATE = 1;
        break;
    }
}
if (STATE)
{
    StartGame();
    thread t1(ThreadFunc);
    HANDLE handle_t1 = t1.native_handle();

    while (true)
    {
        temp = toupper(_getch());
        if (STATE == 1)
        {
            if (win == 9)
            {
                PauseGame(handle_t1);
                ExitGame(t1);
                return 0;
            }
            else if (temp == 'P')
            {
                PauseGame(handle_t1);
            }
            else if (temp == 'L')
            {
                PauseGame(handle_t1);
                SaveData();
                ExitGame(t1);
                return 0;
            }
            else if (temp == 'T')
            {
                PauseGame(handle_t1);
                LoadData();
                ResumeThread(handle_t1);
            }
            else if (temp == 27)
            {
                ExitGame(t1);
                return 0;
            }
            else
            {
                if ((temp != CHAR_LOCK) && (temp == 'D' || temp
== 'A' || temp == 'W' || temp == 'S'))
                {
                    MOVING = temp;
                    ResumeThread(handle_t1);
                }
            }
        }
    }
}
return 0;
}

```

- **Hàm void drawbox()**

Code
<pre>void GotoXY(int x, int y) { COORD coord; coord.X = x; coord.Y = y; SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord); }</pre>

- **Hàm void FixConsoleWindow()**

Hàm được bổ sung thêm hàm MoveWindow() nhận vào tham số consoleWindow... và chiều dài, chiều rộng để cố định kích thước của màn hình (tham khảo một số trên StackOverflow).

Code
<pre>void FixConsoleWindow() { RECT ConsoleRect; HWND consoleWindow = GetConsoleWindow(); GetWindowRect(consoleWindow, &ConsoleRect); LONG style = GetWindowLong(consoleWindow, GWL_STYLE); style = style & ~(WS_MAXIMIZEBOX) & ~(WS_THICKFRAME); SetWindowLong(consoleWindow, GWL_STYLE, style); MoveWindow(consoleWindow, ConsoleRect.left, ConsoleRect.top, 800, 500, TRUE); }</pre>

- **Hàm void InitialGame()**

Hàm này được tạo bằng cách đóng gói một số lệnh của hàm main() lại.

WIDTH_CONSOLE và HEIGH_CONSOLE là hai biến được tính toán hợp lý để lưu chiều dài và rộng của màn hình game. Info.bVisible = FALSE để ẩn con trỏ nháy input trên màn hình console (tham khảo trên StackOverflow). Ta gán STATE = 0 để xác định rằng game chưa được chơi.

Code
<pre>void InitialGame() { system("title Snake"); FixConsoleWindow(); CONSOLE_SCREEN_BUFFER_INFO csbi; GetConsoleScreenBufferInfo(GetStdHandle(STD_OUTPUT_HANDLE), &csbi); WIDTH_CONSOLE = csbi.srWindow.Right - csbi.srWindow.Left; HEIGH_CONSOLE = csbi.srWindow.Bottom - csbi.srWindow.Top - 1; HANDLE consoleHandle = GetStdHandle(STD_OUTPUT_HANDLE);</pre>

```

    CONSOLE_CURSOR_INFO info;
    info.dwSize = 100;
    info.bVisible = FALSE;
    SetConsoleCursorInfo(consoleHandle, &info);

    STATE = 0;
}

```

- **Hàm void ResetData()**

Hàm này chỉ khởi tạo/ cập nhật một số giá trị của game như: vòng chơi, sự tồn tại của công, các hướng được đi và không được đi, tốc độ, chỉ mục thức ăn, kích thước rắn...

Code
<pre> void ResetData() { win = 0; ROUND = 1; GATE_EXIST = false; INDEX_ID = 4; CHAR_LOCK = 'A', MOVING = 'D', SPEED = 1; FOOD_INDEX = 0, SIZE_SNAKE = 4; SIZE_PLUS = 0; snake[0] = { WIDTH_CONSOLE / 2 - 2, HEIGH_CONSOLE / 2 }; snake[1] = { WIDTH_CONSOLE / 2 - 1, HEIGH_CONSOLE / 2 }; snake[2] = { WIDTH_CONSOLE / 2, HEIGH_CONSOLE / 2 }; snake[3] = { WIDTH_CONSOLE / 2 + 1, HEIGH_CONSOLE / 2 }; GenerateFood(); } </pre>

- **Hàm void DrawBoard()**

Vẽ các borders của game bằng hai vòng for. Đồng thời vẽ ra các thông số game (Length, Round), cách điều khiển rắn(Move Snake) và phím hướng dẫn chơi (Pause, Save, Load, Exit).

Code
<pre> void DrawBoard(int x, int y, int width, int height) { for (int i = 0; i < width + 1; i++) { GotoXY(x + i, y); cout << (unsigned char)223; GotoXY(x + width - i - 1, y + height - 1); cout << (unsigned char)220; Sleep(0); } for (int i = 0; i < height; i++) { GotoXY(x + width - 19, y + height - i - 1); cout << (unsigned char)219; GotoXY(x, i); cout << (unsigned char)219; Sleep(0); } } </pre>

```

    }
    GotoXY(width - 14, 2);
    cout << "Length: " << SIZE_SNAKE;
    GotoXY(width - 14, 3);
    cout << "Round: " << ROUND;
    char Line[] = "MOVE SNAKE";
    GotoXY(width - 15, 5);
    TextColor(6, Line);
    GotoXY(width - 14, 6);
    cout << "W : Up ";
    GotoXY(width - 14, 7);
    cout << "S : Down ";
    GotoXY(width - 14, 8);
    cout << "D : Right ";
    GotoXY(width - 14, 9);
    cout << "A : Left ";
    GotoXY(0, height + 1);
    cout << "Pause : P \t Save : L \t Load: T \t Exit: Esc";
}

```

- **Hàm void drawbox()**

Code
<pre> void drawbox(int x, int y, int width, int height) { for (int i = x; i < x + width; ++i) { GotoXY(i, y); cout << char(196); GotoXY(i, y + height); cout << char(196); } for (int i = y; i < y + height; ++i) { GotoXY(x, i); cout << char(179); GotoXY(x + width, i); cout << char(179); } GotoXY(x, y); cout << char(218); GotoXY(x, y + height); cout << char(192); GotoXY(x + width, y); cout << char(191); GotoXY(x + width, y + height); cout << char(217); } </pre>

- **Hàm void TextColor()**

Hàm TextColor in nội dung (OutputContent) theo màu (color). Hàm này được tham khảo trên StackOverflow.

Ở dòng 6, hàm GetConsoleScreenBufferInfo lấy thuộc tính màu hiện tại của screen buffer. Dòng 7, hàm SetConsoleTextAttribute thiết lập thuộc tính của các ký tự được viết vào console screen buffer sau đó với màu color. Sau khi xuất OutputContent, ta thiết lập lại thuộc tính màu lightgray ở dòng 9.

Code

```

void TextColor(int color, char* OutputContent)
{
    static int __BACKGROUND;

    HANDLE h = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_SCREEN_BUFFER_INFO csbiInfo;

    GetConsoleScreenBufferInfo(h, &csbiInfo);

    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color +
(__BACKGROUND << 4));

    cout << OutputContent;

    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), 7 +
(__BACKGROUND << 4));
}

```

• *Hàm void ExitGame()*

Hàm này được hiệu chỉnh lại. Vì TerminateThread() buộc thread kết thúc không an toàn nên sau khi chạy xong thì chương trình báo lỗi. Do đó hàm ExitGame() được cải thiện bằng cách truyền tham chiếu thread& t, rồi gọi t.detach() để gộp luồng, kết thúc chương trình.

Code
<pre> void ExitGame(thread& t) { system("cls"); t.detach(); } </pre>

10. Nguồn tham khảo

- https://courses.ctda.hcmus.edu.vn/pluginfile.php/77504/mod_resource/content/1/Project.pdf
- <https://github.com/phduong2001/snake>
- <https://github.com/nguyenvanhieuvn/Snake>
- https://www.youtube.com/results?search_query=hunting+snake+hcmus
- <https://freesound.org/search/>

HẾT./.